Actionable Cyber Threat Intelligence using Knowledge Graphs and Large Language Models

1st Romy Fieblinger
Mittweida University of Applied Sciences
Mittweida, Germany
rfieblin@hs-mitweida.de

2nd Md Tanvirul Alam
Rochester Institute of Technology
Rochester NY, USA
ma8235@rit.edu

3rd Nidhi Rastogi
Rochester Institute of Technology
Rochester NY, USA
nidhi.rastogi@rit.edu

2024 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW) | 979-8-3503-6729-4/24/$31.00 ©2024 IEEE | DOI: 10.1109/EUROSPW61312.2024.00018

***Abstract*—Cyber threats are constantly evolving. Extracting actionable insights from unstructured Cyber Threat Intelligence (CTI) data is essential to guide cybersecurity decisions. Increasingly, organizations like Microsoft, Trend Micro, and CrowdStrike are using generative AI to facilitate CTI extraction. This paper addresses the challenge of automating the extraction of actionable CTI using advancements in Large Language Models (LLMs) and Knowledge Graphs (KGs). We explore the application of state-of-the-art open-source LLMs, including the Llama 2 series, Mistral 7B Instruct, and Zephyr for extracting meaningful triples from CTI texts. Our methodology evaluates techniques such as prompt engineering, the guidance framework, and fine-tuning to optimize information extraction and structuring. The extracted data is then utilized to construct a KG, offering a structured and queryable representation of threat intelligence. Experimental results demonstrate the effectiveness of our approach in extracting relevant information, with guidance and fine-tuning showing superior performance over prompt engineering. However, while our methods prove effective in small-scale tests, applying LLMs to large-scale data for KG construction and Link Prediction presents ongoing challenges.**

***Index Terms*—Cyber Threat Intelligence, Large Language Models, Knowledge Graphs, Threat Prediction**

1. Introduction

The ever-evolving landscape of cyber threats has made incident threat analysis challenging, even for experienced professionals. Cyber Threat Intelligence (CTI) addresses this by providing information on threat actors, including indicators of compromise (IoCs) such as IP addresses or file hashes, as well as attacker tactics, techniques, and procedures (TTPs). This information is crucial for cybersecurity analysts, enabling them to make informed security decisions and stay updated on new threats, per a 2023 CTI survey by SANS Security [1]. However, CTI is primarily provided in an unstructured format and can be noisy, making knowledge extraction labor-intensive, with over half of the security teams spending more than 40% of their time on these tasks [1]. Therefore, we need more automation to reduce the overwhelming volume of open source reporting and the large amount of time spent analyzing it [1]. To address these challenges, methodologies like TTPHunter [2] and LADDER [3] have been proposed to structure CTI reports into Knowledge Graphs (KGs). Despite their contributions, they often face limitations, such as high false positive rates in classifying IoCs and TTPs, scalability challenges, and the potential to miss critical information due to the constraints of language and predefined schemas.

Lately, large language models (LLMs) have proven valuable for understanding and manipulating natural language. With tailored prompts or additional training, they can perform effectively in tasks such as Named Entity Recognition [4], Relation Extraction [5] [6], and Triple Extraction for constructing Knowledge Graphs [7] [8] [9]. Cybersecurity applications include interpreting TTPs [10] and comprehending cybersecurity terminologies [11] [12]. However, LLMs' potential for extracting and interpreting critical information from natural language-heavy CTI remains largely unexplored.

Therefore, in this research, we address the challenge of automatically extracting actionable Cyber Threat Intelligence (CTI) using Large Language Models (LLMs) and Knowledge Graphs (KGs) (see Figure 1). Specifically, we investigate the application of open-source LLMs, including the Llama 2 series [13], Mistral 7B Instruct [14], and Zephyr [15], for extracting triples from CTI text. Our approach includes the evaluation of various techniques like few-shot learning with prompt engineering [16] and the guidance framework [17], as well as fine-tuning [18]. The best-performing model, as determined by the ROUGE score and human evaluation, is then employed to generate a KG from CTI reports. This graph is subsequently applied to link prediction tasks, showcasing the practical implications of our work in enhancing cybersecurity defenses and response strategies.

The paper makes the following key contributions:

1) We investigate using LLMs to extract information from unstructured CTI for KG construction. While prior work used machine learning, we are the first, to our knowledge, to employ LLMs here.
2) We explore this task under few-shot prompting and fine-tuning settings, testing various LLMs, prompts, inference, and decoding strategies. Our study offers practical insights for effective LLM use in this context.
3) We identify LLM limitations for large-scale datasets and propose practical solutions. By highlighting these challenges, we aim to advance LLM applicability and scalability in information extraction.

DOI 10.1109/EuroSPW61312.2024.00018

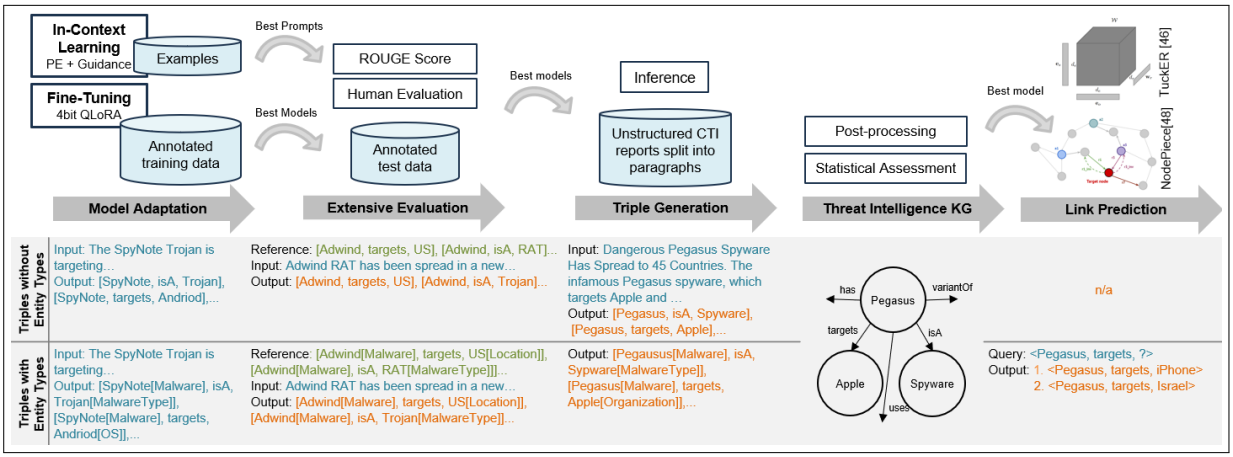

Figure 1: Proposed approach: Outline for LLM-based CTI extraction and KG development. Initial stages involve adapting the models with Few-Shot Prompting and Fine-tuning to generate triple output. Following this, extensive evaluation determines the best model and prompt combination. The top models are then utilized for triple generation in the KG, leading to Link Prediction on the optimal KG, showcasing the transformation from raw data to actionable intelligence.

2. Background and Related Work

Cyber Threat Intelligence (CTI). CTI involves gathering and analyzing security threats information to aid decision-making and help organizations respond effectively [19]. Public CTI sources include threat reports, vendor blogs (e.g., Symantec [20], Mandiant [21], CrowdStrike [22]), news sites, databases, and social media. Effective CTI must be relevant to the organization, actionable for immediate threat response, and contribute to key business outcomes [23]. In recent years, several approaches have emerged for extracting information from CTI reports, primarily focusing on TTPs. Early works like TTPDrill [24] and ChainSmith [25] used NLP to extract threat actions and IoCs. CASIE [26] combined deep learning and linguistic features for cybersecurity event and vulnerability extraction. More recently, transformer-based architectures like SecureBERT [27] and TTPHunter [2] fine-tuned models such as BERT [28] or RoBERTa [29] for better cybersecurity text classification and context-sensitive TTP extraction.

Large Language Models (LLMs). LLMs are transformer-based models [30] that excel in language modeling by estimating the probability of word sequences and generating text. These models undergo pre-training on extensive textual corpora, significantly enhancing their effectiveness in various NLP tasks, such as text generation and machine translation by providing context-aware representations. LLMs are characterized by their large scale, with tens or hundreds of billions of parameters [31]. **Fine-tuning** is commonly applied to models like Llama 2-Chat for task-specific enhancements, employing techniques such as Instruction Fine-tuning, which leverages prompt, response pairs to improve adherence to textual instructions. Due to the high resources required for full fine-tuning of all model layers, Parameter-Efficient Fine-Tuning (PEFT) methods like LoRA [18] and its quantized version, QLoRA [32], which selectively update parameters, have gained popularity. **Prompt Engineering (PE)** is another method to direct language models towards desired outputs by iteratively refining the input for a generative model, circumventing the need for extensive annotated data or altering model parameters. **Few-shot prompting** includes examples in the prompt, giving the model additional context which aids in boosting its performance by guiding the model in generating outputs that mirror the patterns in the examples [33]. LLMs have found diverse applications in the field of cybersecurity. For example, ChatIDS [11] utilizes an LLM to interpret and explain anonymized intrusion detection system alerts to non-experts, providing user-friendly explanations and demonstrating its understanding of cybersecurity concepts. Additionally, Fayyazi et al. [10] explored using LLMs, such as GPT-3.5 and Bard, along with supervised training-based BaseLLMs, to classify cybersecurity descriptions into MITRE ATT&CK tactics. The MITRE ATT&CK framework [34], a freely available knowledge base, categorizes adversary tactics and techniques to aid in understanding and defending against cyber threats. Fayyazi et al. [10] highlighted challenges related to TTP description ambiguity and the importance of precise prompts. While LLMs have recently been successfully applied to entity and relationship extraction in various domains [35] [6] [36] [5], their specific application in extracting structured information from CTI reports within the cybersecurity field has received less attention. Siracusano et al. [37] employed GPT-3.5 for CTI extraction. They implemented zero-shot prompting and in-context learning in two custom pipelines to develop a structured CTI extraction tool, streamlining the manual information extraction process.

Knowledge Graphs (KGs). KGs are structured representations of real-world information in the form of triples $<e_{head}, r, e_{tail}>$, where e_{head} denotes the head entity, e_{tail} the tail entity and r the relationship between the entities [38]. Their ability to model structured, complex data in a machine-readable format makes KGs indispensable across diverse domains such as question-answering or information retrieval [3]. In CTI, KGs structurally represent complex relationships and integrate diverse data sources to enhance semantic querying, data analysis, and decision-making, thereby offering superior situational awareness and predictive insights. The SEPSES Cybersecurity Knowledge Graph [39], for example, combines comprehensive information on attack patterns and vulnerabilities from publically accessible sources into a dynamic KG. Frameworks such as APTKG [40] and AttacKG [41]

focus on the automated extraction and organization of threat intelligence entities and their relationships from CTI reports into KGs, aiming to improve the understanding of attack methods. THREATKG [42] expands on this by automating the collection and integration of open-source threat intelligence into a KG, which is continually updated with new information. These studies highlight KGs' role in organizing CTI for advanced security analytics and forecasting. TINKER [43] and LADDER [3] convert textual attack patterns into MITRE ATT&CK-aligned structured data, further applying link prediction on the resultant CTI KGs.

Link Prediction (LP). LP addresses KG incompleteness by predicting missing facts [38]. It identifies the missing entity in a relationship (head or tail prediction), completing a triplet either as the subject (head prediction) in the format $<?, r, e_{tail}>$ or as the object (tail prediction) in $<e_{head}, r, ?>$, effectively completing the given triplet in the KG. KG embedding techniques like RotatE [44], ConvE [45], and TuckER [46] (used in tools like TINKER [43] and LADDER [3]) have proven successful on benchmarks. LP can be performed in a transductive setting, using only entities seen during training, or in an inductive setting, which accommodates unseen entities during testing through additional inference statements. [47]. While TuckER [46] is designed for transductive LP, newer methods like NodePiece [48] are equipped to handle both settings. Link prediction in cybersecurity KGs enables proactive threat detection by forecasting potential future connections, like new malware attack patterns [3]. It aids vulnerability assessment by revealing possible attack paths, helping organizations strengthen their defenses. It also improves incident response and risk management by pinpointing and prioritizing potential threat vectors [49].

3. Approach

3.1. Research Questions

We address the following research questions (RQs) in this study through extensive experiments and analysis:

1) **RQ1(Few-Shot):** How effective are LLMs at extracting CTI information in a few-shot setting? *For this*, we investigate prompt engineering & guidance frameworks [17].
2) **RQ2 (Fine-tuning):** How much does fine-tuning LLMs with labeled data improve CTI information extraction? *For this*, we employ the parameter-efficient QLORA method.
3) **RQ3 (Knowledge Graph Quality):** What is the quality of triples generated from a large CTI corpus using LLMs, and how can we improve them? *For this*, we identify shortcomings and propose error reduction methods.
4) **RQ4 (Link Prediction):** How does CTI-derived knowledge graph perform in link prediction? *For this*, we consider both transductive and inductive settings.

3.2. Pretrained Models

We selected several open-source pretrained LLMs based on their diverse capabilities. The Llama 2 series (7B, 7B chat, 13B, 13B chat, 70B chat) [13], Mistral 7B Instruct v2.0 [14], and Zephyr-7B-β [15] were included in the analysis (summary in Table 1). The Llama 2 series, introduced by Meta AI in February 2023, offers a range of pre-trained and fine-tuned LLMs with capacities ranging from 7 to 70 billion parameters. This variety enables a comprehensive analysis of techniques at different scales [13]. Notably, the Llama 2 70B model outperforms MosaicML Pretrained Transformer (MPT) and Falcon [13]. Llama 2-Chat models are fine-tuned with supervised fine-tuning and further trained with reinforcement learning with human feedback (RLHF) to align model behavior with human preferences and instruction following. They exhibit strong temporal knowledge organization abilities even with limited data, and its 70B version surpasses ChatGPT in answering factual questions [13]. Mistral 7B Instruct, fine-tuned on instruction datasets from the Hugging Face repository, outperforms smaller Llama 7B and 13B versions [14]. Additionally, Zephyr, based on Mistral 7B and fine-tuned with distilled Direct Preference Optimization (dDPO) to improve intent alignment, outperforms Llama 70B chat in MT-Bench tests [15].

TABLE 1: Summary of Selected Models

Model	Size	Version	Architecture	Align
Llama 2	7B	Base	Enhanced Llama 1 with doubled context length and expanded pre-training data	-
	7B	Chat		RLHF
	13B	Base		-
	13B	Chat		RLHF
	70B	Chat	Enhanced Llama 1 with doubled context length, expanded pre-training data, and GQA	RLHF
Mistral	7B	Instruct v2.0	GQA	-
Zephyr	7B	β	Fine-tuned version of Mistral 7B v0.1	dDPO

3.3. Dataset Generation

We utilized two datasets, introduced by Alam et al. in [3], for our extraction experiments with LLMs: a manually annotated dataset for model training and evaluation and a large-scale dataset for knowledge graph (KG) generation and link prediction. The fine-tuning and evaluation dataset contains 120 curated CTI reports related to 36 Android malware families between the years 2015 and 2022. The reports were manually annotated using the BRAT annotation tool, capturing cyber threat concepts such as `Malware`, `Malware Type`, `Application`, `Operating System`, `Organization`, `Person`, `Time`, `Threat Actor`, `Location`, `Indicator`, and `Attack Pattern`. These concepts are connected by ten relations: `isA`, `targets`, `uses`, `hasAuthor`, `hasAlias`, `indicates`, `discoveredIn`, `exploits`, `variantOf`, `has`.

To accommodate the model's maximum content length of 4096 tokens and address the limited number of examples, we divided the text from annotated CTI reports into paragraphs. Various dataset sizes were tested, ranging from 400 to 1000 tokens per training example, resulting in different numbers of training examples. For example, for 400 tokens, the data was divided into 909 paragraphs. The annotated triples were matched automatically with the corresponding paragraphs via a script. However, not all paragraphs contained content that could be directly associated with the annotated triples. As a result, only 768 paragraphs successfully matched triples and were included in the final dataset. The dataset was split into training, validation, and test sets in an 80:16:4 ratio, yielding 574

training, 115 validation, and 29 test examples, each with up to 400 tokens per paragraph.

The best-performing LLMs, specifically the fine-tuned Llama 2 7B chat and the Llama 70B chat guidance model, were employed to generate a KG from the second, larger dataset comprising approximately 12,000 unstructured open-access CTI reports. These reports were divided into paragraphs, each limited to a maximum of 1,000 tokens, resulting in around 80,000 paragraphs. These paragraphs were then processed by the LLM to generate triples for the KG.

3.4. Information Extraction Methods using LLM

3.4.1. Few-Shot Setting. Two methods for extracting triples under a few-shot setting with LLMs are explored – prompt engineering and the guidance framework.

Prompt Engineering. To explore the effectiveness of prompt engineering, experiments were conducted with various chat variants of the selected models, including Llama 2 7B chat, 13B chat, and 70B chat, as well as Mistral 7B Instruct v2.0 and Zephyr-7B-β. Prompts were tailored to test the model's adaptability across varying instruction styles and complexity, covering scenarios from zero to few-shot inference with diverse formats and content. We followed the prompt engineering guidelines from OpenAI [16] [50], which contain best practices on how to give clear and effective instructions and share strategies and tactics for getting better results from LLMs. For example, including the ontology and instruction in the system prompt `[INST] <<SYS>>\nExtract cybersecurity-related triples consisting of entities of the types Malware, Malware Types, Applications, Operating Systems, Organizations, Persons, Times, Threat Actors, Locations, and Attack Patterns and relationships between these entities of the types isA, targets, uses, hasAuthor, hasAlias, indicates, discoveredIn, exploits, variantOf, and has. Print the extracted triples in the format: [Entity1, Relation, Entity2]\n<<SYS>>\n\nExtract triples from the following text:‘‘{input_txt}’’ [/INST]` compared to only including the relationships or no information about the ontology in the prompt, like `[INST] What are the [subject, predicate, object]-triples in the following text? Text:‘‘{input_txt}’’ [/INST]`. Additionally, the number and format of the included examples, such as `Input text: ‘‘A new version of the SpyNote Trojan is designed to trick Android users into thinking it’s a legitimate Netflix application. Once installed, the remote access Trojan (RAT) essentially hands control of the device over to the hacker, enabling them to copy files, view contacts, and eavesdrop on the victim, among other capabilities.‘‘`

`Extracted triples: [SpyNote, isA, Trojan], [SpyNote, targets, Andriod], [SpyNote, uses, designed to trick Android users into thinking it’s a legitimate Netflix application], [SpyNote, isA, remote access Trojan], [SpyNote, isA, RAT], [SpyNote, uses, hands control of the device over to the hacker], [SpyNote, uses, enabling them to copy files], [SpyNote, uses, view contacts], [SpyNote, uses, eavesdrop on the victim]` varied between zero to four, separated by only line breaks, or the predefined prompt format, e.g., for Llama 2 `Input text: {ex_txt} [/INST] Extracted triples: {ex_triples} </s><s>[INST] Input text: {text} [/INST]`.

Guidance. Guidance [17] is a Python library by Microsoft Research for controlling LLM output, using a special guidance language for applying constraints like regular expressions or Context-free grammar (CFGs). It constructs an Abstract Syntax Tree (AST) for program interaction, customizing LLM responses and stopping points. [51]. Additionally, the guidance framework boosts efficiency and accuracy by batching user-added text and omitting non-essential parts [17].

In this research, the *guidance* library is employed to enforce the desired output format for the Llama 2-Chat models. Its capability to ensure data structure compliance and seamless integration into data processing tasks makes it a valuable tool. Using a regular expression such as `\\n|</s>` as a stopping criterion makes the generation stop upon encountering either a line break or an end-of-sentence token generated by the LLM. However, guidance currently lacks support for negative or positive lookaheads and generates parsing errors in regular expressions, especially with special operator symbols like the closing bracket `]`, greatly limiting its potential applications. Consequently, in our experiments, only the stop regular expression is effectively utilized. Instructions and examples can be incorporated as follows, once the model has been loaded and referenced as `llama2`:

```
def instruction(lm, ex_txt, ex_triple, input_txt):
    lm += f'''Extract triples from the following input text. Your answers
        need to be in the format [subject, predicate, object].
    ----------
    Input text: {example_txt}
    Extracted triples: {example_triple}
    ----------
    Input text: {input_txt}
    Extracted triples: '''
    return lm

output = llama2
        + instruction(ex_txt, ex_triples, input_txt)
        + gen(name="triples", temperature=0.6, regex=r'.+',
              max_tokens=2048, stop_regex='\\n|</s>')
```

3.4.2. Fine-tuning. While prompt engineering can yield significant improvements by optimizing prompts, fine-tuning a language model can enhance outcomes in specific scenarios and increase robustness. Therefore, a subset of the experiments focused on instruction fine-tuning the Llama 2 models (7B, 7B chat, 13B, and 13B chat) using a specific training dataset described in section 3.3. QLoRA with 4-bit quantization was implemented to fine-tune the models in a resource-efficient manner. The experiments encompassed several factors, including different prompts, dataset sizes (ranging from 357 paragraphs with 1000 tokens each to 768 paragraphs with 400 tokens each), training configurations (using linear, constant, or cosine learning schedulers with learning rates of 2e-4 or 2e-5), and LoRA parameters (with ranks and alphas of 8, 16, 32, or 64), allowing for a comprehensive exploration of the fine-tuning process. The prompts experimented with various text separations from the instructions, employing line breaks, quotes, and instructional symbols such as `<txt></txt>` and `[TXT][/TXT]`, or using no separation at all. Additionally, the complexity and length of the prompt were varied, ranging from the inclusion of comprehensive ontology information, e.g., `[INST] Extract cybersecurity-related triples from the following text. Present them in the structure [subject, predicate, object].`

`The following concepts should be used for subjects/objects: Malware, Malware Type, Application, Operating System, Organization, Person, Time, Threat Actor, Location, and Attack Pattern. The concepts should be connected through the following ten relationships as predicate: isA, targets, uses, hasAuthor, hasAlias, indicates, discoveredIn, exploits, variantOf, and has. The input text is: ''{input_txt}'' [/INST]` to prompts with only a brief task instruction, like `[INST] Extract [subject, predicate, object]-triples from the following input text. Input text: ''{input_txt}'' [/INST]`. For fine-tuning the base models, the commonly used Alpaca prompt format [52] was employed in the form: `### Instruction: Extract [subject, predicate, object]-triples from the following input text.\n### Input: {input_txt}\n### Response: {triples}`

3.4.3. Inference & Decoding Strategies. Transformer models predict next token by producing a probability distribution across all possible words, where decoding strategies crucially impact text coherence and repetition. Greedy decoding risks repetition by choosing the likeliest next token, while beam-search considers multiple paths for higher-quality text [53]. Beam-search multinomial sampling blends the randomness of multinomial sampling with beam-search's strategy to optimize text generation [54]. Adjusting decoding strategies and parameters like repetition penalty can improve text quality without changing the model's core settings. The mentioned decoding strategies have been tested for prompt engineering and fine-tuning in the context of structuring CTI. The *guidance* library currently does not support beam-search decoding or sampling strategies.

3.5. Implementation Details

To handle the computational demands of these large models, the experiments utilized NVIDIA A100 Tensor Core GPUs within a SPORC (Scheduled Processing On Research Computing) High-Performance Computing (HPC) cluster.

The models were loaded from Hugging Face [55] using the *transformers* library [56]. For the quantization of smaller models, *bitsandbytes* [57] was utilized, whereas the Llama 2 70B chat model was used in its quantized form with *autogptq* [58]. The *peft* [59] and *trl* [60] library were employed for LoRA fine-tuning, and applied to all linear layers. Additionally, *Accelerate* [61] was used to enhance inference speed during KG generation. For comparison, the maximum token length was set to 2048 tokens, which proved sufficient given the small input size of 1000 tokens. The temperature and repetition penalty for all models were set at 0.6 and 1.1, respectively. Decoding strategies and few-shot examples are detailed in Table 2.

TABLE 2: Overview of LLM Inference Parameters: The context length is set at 2048 tokens for all models, the temperature is set to 0.6, and repetition penalty to 1.1.

Technique	Model	Decoding Strategy	E.g. Number
PE	Llama 2 models	Multinomial Sampling	1-2
	Mistral 7B Instruct	Greedy	1
	Zephyr-7B-β	Greedy	2
Guidance	Llama 2 models	Greedy	1-2
Fine-tuning	Llama 2 models	Beam-search	0

3.6. Evaluation Metrics

Traditional evaluation metrics, such as Accuracy and F1-score, where only exact matches are compared, are inadequate for assessing the performance of generative models. Since these models generate new content based on learned patterns, they require more nuanced metrics that capture the quality and relevance of the generated text. We adopt the ROUGE metrics [62] in our study to evaluate the quality of triples generated by LLMs. **ROUGE** (Recall-Oriented Understudy for Gisting Evaluation) [62] comprises a suite of metrics designed to compare generated text with reference text, making it suitable for applications where such comparisons are feasible, including translation, text summarization, and entity extraction. The effectiveness of ROUGE heavily depends on the quality of the reference text; poor-quality references can lead to misleading assessments of the generated content's quality. It is case-insensitive and produces values ranging from 0 to 1. ROUGE-N evaluates the overlap of n-grams (unigrams, bigrams, and trigrams) between a text and its reference, using metrics like precision, recall, and F1-score. Recall measures how many relevant words from the reference text are captured by the machine-generated text, emphasizing the inclusion of all pertinent information. The recall of n-gram overlaps between a candidate text and a set of reference texts is calculated as follows:

$$\text{ROUGE-N}_{recall} = \frac{\sum_{S \in \{\text{Reference}\}} \sum_{gram_n \in S} \text{Count}_{\text{match}}(gram_n)}{\sum_{S \in \{\text{Reference}\}} \sum_{gram_n \in S} \text{Count}(gram_n)} \tag{1}$$

Precision assesses the proportion of words in the machine-generated text that accurately match those in the reference text, focusing on the relevance and correctness of the information provided. It is calculated with the following formula:

$$\text{ROUGE-N}_{precision} = \frac{\sum_{S \in \{\text{Reference}\}} \sum_{gram_n \in S} \text{Count}_{\text{match}}(gram_n)}{\sum_{S \in \{\text{Canidate}\}} \sum_{gram_n \in S} \text{Count}(gram_n)} \tag{2}$$

The F1-score balances recall and precision, offering a single metric to assess overall performance:

$$\text{F}_1\text{-score} = \frac{2 * P * R}{P + R} \tag{3}$$

Considering the reference hand-annotated triple `[Adwind, targets, retail and petroleum industry]` and the candidate `[Adwind, targets, petroleum production industry]` extracted with the LLM, the ROUGE-1 recall would be $4/6 = 0.667$, representing four unigram matches out of six possible unigrams in the reference. Subsequently, the precision would be calculated as $4/5 = 0.8$, and F1-score $2 * 0.667 * 0.8/1.467 = 0.727$, which is reported in this paper. It is important to note that for ROUGE-N the order in which different n-grams appear in the text does not influence the score. However, the words within an individual n-gram must be consecutive. For example, in ROUGE-2, the matches would only include `[Adwind, targets, retail and petroleum industry]` from the reference and `[Adwind, targets, petroleum production industry]` from the candidate. In contrast, ROUGE-L assesses the longest common subsequence between texts, capturing similarity in content without requiring consecutive word matches but in-sequence matches. This allows ROUGE-L to reward summaries

with similar content even if their sentence structures differ. Thus, in the given example, the reference and candidate would be matched as `[Adwind, targets, retail and petroleum industry]` and `[Adwind, targets, petroleum production industry]`, respectively.

To evaluate the model's performance on the test set, triples were extracted from the LLM's output using regular expressions and sorted alphabetically. The hand-annotated triples from the dataset, also sorted alphabetically, served as the reference text for comparison. For the initial assessment, a manually evaluated example paragraph was used. Based on the desired outcome, the prompt and settings that produced the closest alignment with the desired output were applied to generate outputs for the remaining 29 examples in the test set. The generated results were evaluated using ROUGE metrics [62] and human evaluation.

4. Experiment and Results

4.1. Few-Shot Performance (RQ1)

Prompt engineering significantly affects the output of LLMs, with elements such as prompt structure, spacing, line breaks, and punctuation playing a crucial role. The impact of the prompt design was evident when comparing outputs from different sizes of the Llama 2 model; a prompt that was highly effective with the 7B model yielded less satisfactory results with the 13B variant. For example, considering the one-shot prompt `[INST] <<SYS>>\nExtract [subject, predicate, object]-triples from the input text with the following predicates: isA, targets, uses, hasAuthor, hasAlias, indicates, discoveredIn, exploits, variantOf, and has.\n<<SYS>>\n\nInput text: {ex_txt} [/INST] Extracted triples: {ex_triples} </s><s>[INST] Input text: {input_txt} [/INST] Extracted triples:` . This prompt yielded the desired output as illustrated in Table 5, where prompt engineering combined with greedy decoding produced the expected results for the 7B model. However, applying the same configuration to the 13B model resulted in merely listing entities from the text, e.g., `The Adwind RAT, petroleum industry, US, new campaign, multi-layer obfuscation, evading detection,...`. Consequently, this makes it a time-consuming process of trial and error. While adding a single example often enhanced the output, incorporating multiple examples, as the one mentioned in section 3.4.1, did not necessarily lead to better outcomes and, in some instances, even deteriorated the model's performance. For instance, the Llama 2 7B chat model produced continuous text with over two examples, whereas other models struggled with inaccurately inventing relationships like `hostedOn` or `usedFor` from text verbs. Llama 2-Chat models presented notable challenges in generating outputs in a format suitable for further processing, in contrast to Mistral and Zephyr models. Llama models exhibited less consistency in output formatting, whereas Mistral, in particular, was able to produce the desired format reliably, even in zero-shot inference scenarios. The prompt `[INST] <<SYS>>\nYou are a helpful cyber-security assistant. Your task is to extract [entity,relationships,entity]-triples from the input text. Use only the relationships: 'isA', 'targets', 'uses', 'hasAuthor', 'hasAlias', 'indicates', 'discoveredIn', 'exploits', 'variantOf', and 'has'. Your answers need to be in the format [entity,relationship,entity]. Reply only with the extracted triples.\n<</SYS>>\n\nWhat are the triples in the following input text: {input_txt} [/INST]` with Mistral greedy decoding already yielded triples in the format `[Adwind RAT,isA,Remote Access Trojan (RAT)], [Adwind RAT,targets,petroleum industry], [Adwind RAT,targets,US]`.

To ensure the model maintained the desired relationship types in the triples, merely providing examples was insufficient; incorporating the ontology directly into the prompt proved to be crucial. The most effective results across all models were achieved by combining the ontology with one or two examples in the prompt, as illustrated in the prompt described in the first paragraph of this section.

Our comparative analysis of model performance with prompt engineering is summarized in Table 3, which presents the ROUGE metric scores in the context of n-gram overlap for n=1,2,3,6 (ROUGE-N), as well as the longest common subsequence (ROUGE-L) for different prompts and models applied to the extraction of triples from the test data. It lists the results of the best-performing prompts for the models following the triple extracting from the LLM's output. The ROUGE scores support the findings that Mistral and Zephyr outperform Llama 2 models when only prompt engineering is applied. The frequent failure of the Llama models to generate the desired triples led to a score of 0 for those specific test instances, which significantly reduced the overall ROUGE score. Noteworthy is the performance of the Llama 2 70B chat model, which, when prompted using a one-shot technique, achieved results that were the most comparable. However, its lower score can be attributed to its inability to fully comply with the specified ontology, particularly concerning the relationship types outlined in the prompt. This led to the generation of triples that diverged from the manually annotated reference, a discrepancy that became more pronounced in longer texts.

TABLE 3: Comparison of PROMPT ENGINEERING Model Performance on ROUGE Metrics: Evaluating Various Prompts and Models for Triple Extraction from Test Data. The highest value in each metric is in bold.

Model	Prompt	ROUGE-1	ROUGE-2	ROUGE-3	ROUGE-6	ROUGE-L
Llama 2 7B chat	few-shot	0.3328	0.2012	0.1199	0.0357	0.2407
Llama 2 13B chat	one-shot	0.3825	0.2336	0.1452	0.0450	0.2910
Llama 2 13B chat	few-shot	0.4205	0.2505	0.1521	0.0521	0.3313
Llama 2 70B chat	one-shot	0.3930	0.2414	0.1561	0.0674	0.3040
Mistral 7B Instruct	one-shot	0.4775	**0.3170**	**0.2083**	**0.0833**	**0.3388**
Zephyr-7B-β	few-shot	**0.4332**	0.2856	0.1968	0.0796	0.3209

Guidance demonstrated remarkable efficiency, delivering impressive outcomes and proving easy to implement. It surpassed prompt engineering in achieving higher ROUGE scores, as illustrated in Table 4, which displays the scores for various models using *guidance*. For the smaller Llama 2-Chat models, optimal performance was achieved using the prompt `Extract triples from the following input text. Your answers need to be in the format [entity1, relation, entity2].` with three examples, whereas the 70B chat model showed its best performance with a more extensive prompt that included the ontology, i.e., `Extract [entity, relationships, entity]-triples from the following input text. Only use the`

relationships: ‘isA’, ‘targets’, ‘uses’, ‘hasAuthor’, ‘hasAlias’, ‘indicates’, ‘discoveredIn’, ‘exploits’, ‘variantOf’, and ‘has’. and just one example. Although the 13B chat model attained the highest ROUGE scores, manual human evaluation of the test data’s generated triples exposed its difficulty with complex syntactical structures such as negations, resulting in the majority of triples being incorrect. For instance, from the text `First Twitter-controlled Android botnet discovered Detected by ESET as Android/Twitoor, this malware is unique because of its resilience mechanism. Instead of being controlled by a traditional command-and-control server, it receives instructions via tweets.` the model incorrectly extracts triples such as `[Twitoor, isA, command-and-control server]` and `[Twitoor, isA, traditional command-and-control server]`, failing to correctly interpret the word `instead`.

The same issue was observed with the 7B model, positioning the 70B model as the top-performing guidance model according to human analysis.

TABLE 4: Comparison of GUIDANCE Model Performance on ROUGE Metrics: Evaluating Various Prompts and Models for Triple Extraction from Test Data. The highest value in each metric is in bold.

Model	Prompt	ROUGE-1	ROUGE-2	ROUGE-3	ROUGE-6	ROUGE-L
Llama 2 7B chat	few-shot	0.4724	0.3025	0.1930	0.0635	0.3437
Llama 2 13B chat	one-shot	**0.5263**	**0.3827**	**0.2898**	**0.1516**	**0.4355**
Llama 2 70B chat	one-shot	0.5147	0.3730	0.2791	0.1425	0.3999
Llama 2 70B chat	few-shot	0.5126	0.3527	0.2493	0.1137	0.3905

Finding 1: Guidance performs better than simple prompt engineering in a few-shot setup for CTI triple extraction and requires less effort to achieve accurate outputs.

4.2. Fine-tuning Performance (RQ2)

Fine-tuning with fewer than the 718 examples in the dataset proved insufficient. The most effective approach utilized a constant learning scheduler with a learning rate of 2e-4, and LoRA parameters set with an alpha and rank of 16 for all linear layers. Significant variations in output were observed when different symbols were used to separate the input text from the prompt. Introducing line breaks into the input text for fine-tuning the model led to poor results, characterized by outputs that were either mere numbers or repetitive sections of the prompt. This underscores the critical importance of strict adherence to formatting, particularly regarding line breaks and prompt structure, to attain the intended output. Moreover, longer instructions tended to confuse the language model. For instance, the prompt `[INST] Extract [subject, predicate, object]-triples from the following input text. Input text: ‘‘{input_txt}’’ [/INST]` significantly outperformed more detailed prompts that included the ontology, as discussed in section 3.4.2. The latter failed to extract any triples, merely generating repetitive text regardless of the decoding strategy employed.

As illustrated in Table 5, which presents a comparison of the different outputs generated by applying various decoding strategies in the context of Llama 2 7B chat prompt engineering versus fine-tuned model, the impact on the fine-tuned model is more pronounced. While greedy decoding restricted the output to only one word and multinomial sampling to only two triples with the fine-tuned model, beam-search multinomial sampling resulted in greater variance but highly repetitive output towards the end, even with a repetition penalty of 1.1. Generally, beam-search outperformed other strategies for fine-tuning, while prompt engineering achieved the desired output with greedy decoding and multinomial sampling, as evident in the table with the most relevant triples generated for those strategies. To avoid repetitive output, setting a repetition penalty of 1.1 for beam-search was necessary.

Fine-tuning the base models resulted in repetitive output that did not align with the instructions or the desired outcome, suggesting that the dataset size was insufficient for instruction-based tuning of the base Llama 2 models. Notably, loading the trained LoRA weights from the base models into the chat version for inference yielded good results, comparable to fine-tuning the chat versions directly. This suggests that chat-optimized models, being more adept at handling instructional tasks, can effectively capture essential patterns or knowledge relevant to this application through the LoRA weights, even if they were trained using a different prompt format.

The ROUGE scores for the Llama 2 7B and 13B chat versions, differentiated by the use of either `<txt>` and `</txt>` markers or quotes to separate the input text from the prompt, alongside the fine-tuned base models employing the alpaca prompt format, are presented in Table 6. The fine-tuned chat models demonstrate notable consistency across the metrics, achieving higher scores than any other techniques could attain, highlighting the effectiveness of fine-tuning in improving the quality of model outputs. Manual analysis of the generated triples from the test data supports the ROUGE scores. The fine-tuned Llama 2 7B chat model, utilizing input text marked by `<txt>` and `</txt>` , along with the fine-tuned Llama 2 7B and 13B models loaded onto their respective chat versions, exhibited the best performance. However, the latter models each failed to produce a response for different test samples.

Given that the fine-tuned 7B chat model demonstrated superior performance in both the ROUGE metrics and manual analysis, it was selected for the subsequent phase: generating a KG from CTI data.

Finding 2: Fine-tuning improves over few-shot prompting as evidenced by improved ROUGE score. Shorter prompts outperform longer ones and the separation format between instruction and input text significantly affects performance.

4.3. Knowledge Graph & Enhancement (RQ3)

Despite the model’s optimal performance on the test data, as evidenced by both ROUGE scores and manual human evaluations, its application to a large-scale dataset for KG generation produced very noisy triples. In the initial approach, which involved generating triples from the text without naming entity types, the fine-tuned 7B model extracted 188,547 triples from nearly 12,000 documents. Upon first manual inspection, it became evident that the model generated numerous nonsensical triples. It not only extracted entities that might be correct but did not correspond to a given entity type, such as `[+44113320****, indicates, Phone Numbers]`, but also identified non-named

TABLE 5: Comparison of Decoding Strategies: Llama 2 7B chat Prompt Engineering compared with Fine-tuned Model Across Various Decoding Techniques

Decoding Strategy	Prompt Engineering	Fine-tuned Model
Greedy	[Adwind RAT, isA, RAT], [Adwind RAT, targets, petroleum industry in the US], [Adwind RAT, uses, multi-layer obfuscation to evade detection], [Adwind RAT, isA, malware], [Adwind RAT, has, persistence through registry modifications], [Adwind RAT, performs, process injection to stay under the radar], [Adwind RAT, terminates, security services (e.g., firewall, AV)], [Adwind RAT, steals, sensitive data]	nobody
Beam-search	1. Adwind RAT, isA, malware 2. Adwind RAT, targets, petroleum industry in the US 3. Adwind RAT, uses, multi-layer obfuscation to evade detection 4. Adwind RAT, is hosted on a serving domain ... 10. Adwind RAT, steals sensitive data.	['Adwind', 'targets', 'petroleum industry'], ['Adwind', 'targets', 'US'], ['Adwind', 'targets', 'retail'], ['Adwind', 'targets', 'hospitality'], ['Adwind', 'uses', 'achieves persistence through registry modications'], ['Adwind', 'uses', 'performs process injection to stay under the radar'], ['Adwind', 'uses', 'terminates security services (e.g., rewall, AV)'], ['Adwind', 'uses', 'steals sensitive data']
Multinomial Sampling	[Adwind RAT, isA, malware] [Adwind RAT, targets, petroleum industry in the US] [Adwind RAT, uses, multi-layer obfuscation to evade detection] ... [Adwind RAT, uses, steals sensitive data]	['Adwind', 'targets', 'United States'], ['Adwind', 'isA', 'RAT']
Beam-search Multinomial Sampling	1. Adwind RAT, isA, malware 2. Adwind RAT, targets, petroleum industry in the US 3. Adwind RAT, uses, multi-layer obfuscation to evade detection 4. Adwind RAT, is hosted on a serving domain ... 10. Adwind RAT, steals sensitive data.	['Adwind', 'targets', 'petroleum'], ['Adwind', 'targets', 'US'], ['Adwind', 'targets', 'retail'], ['Adwind', 'targets', 'hospitality'], ['Adwind', 'uses', 'achieves persistence through registry modications'], ['Adwind', 'uses', 'performs process injection to stay under the radar'], ['Adwind', 'uses', 'steals sensitive data'], ['Adwind', 'targets', 'US'], ['Adwind', 'targets', 'petroleum'], ['Adwind', 'targets', 'retail'], ['Adwind', 'targets', 'hospitality'], ...repeatedly

TABLE 6: Comparison of FINE-TUNED Model Performance on ROUGE Metrics: Evaluating Various Fine-tuning Prompt Formats and Models for Triple Extraction from Test Data. The highest value in each metric is in bold, with the second and third highest values underlined.

Model	Prompt	ROUGE-1	ROUGE-2	ROUGE-3	ROUGE-6	ROUGE-L
Llama 2 7B chat	txt	**0.6264**	**0.5419**	**0.4866**	**0.3404**	**0.5667**
Llama 2 7B chat	quotes	0.3976	0.3299	0.2749	0.1696	0.3508
Llama 2 7B on chat	alpaca	<u>0.5965</u>	0.5060	<u>0.4424</u>	0.2842	<u>0.5469</u>
Llama 2 13B chat	txt	0.5786	<u>0.5107</u>	0.4522	<u>0.3162</u>	0.5353
Llama 2 13B chat	quotes	0.5130	0.4443	0.3898	0.2761	0.4678
Llama 2 13B on chat	alpaca	<u>0.6095</u>	<u>0.5103</u>	0.4389	<u>0.2868</u>	0.5345

entities, like `[124, indicates, Android]`, and produced factually incorrect triples, for instance `[/facebook, targets, Facebook]`. A qualitative assessment of 100 randomly sampled triples revealed that only 39 of them were correct. Post-processing, for example, enforcing dates as the only valid subject for the `discoveredIn` relationship, or excluding numbers as subjects, was minimally effective and did not yield a usable KG. This scenario illustrates that while LLMs may perform well on specific, curated test datasets, they struggle with large-scale, primarily unprocessed data. The limitations observed could be attributed partly to the relatively small size of the LLM, having only 7 billion parameters, and possibly to insufficient training data.

To facilitate data analysis of the generated triples and perform post-processing steps in relation to the ontology, additional experiments were conducted not only to extract the triples but also to identify the entity types within them. For instance, triples were generated in formats such as `[Adwind[Malware], targets, US[Location]]` and `[Adwind[Malware], isA, Trojan[MalwareType]]`

Table 7 summarizes the ROUGE scores for various techniques and models that showed promise in earlier stages by producing triples with entity types for the test data. The 70B chat model failed to generate the triples in a format suitable for automatic processing in 19 of the 29 test examples, resulting in a low overall ROUGE score. Mistral and Zephyr achieved significantly better scores; however, manual analysis revealed that, although they extracted numerous triples, they frequently introduced incorrect relationships, such as `hasCapability` or `hasFeature`, and incorrect entity types, such as `Command` or `Attack`. Moreover, there were numerous errors in the entity types for the extracted triples, such as categorizing `APK` as `Indicator` and `banking apps` as `AttackPattern`.

The guidance models attained even higher ROUGE scores, but human evaluation of the generated indicated that they struggled with accurately extracting and classifying indicators, often misclassifying them as `AttackPattern`. Furthermore, when the extracted entity was correctly identified as `Indicator`, the relationship often did not adhere to the ontology, for example `[FakeSpy[Malware], isA, ANDROIDOS_LOADGFISH.HRX[Indicator]]`. The guidance models also struggled with adhering strictly to predefined relationships.

The fine-tuned model was most accurate in adhering to the ontology, especially with `AttackPattern` and `Indicator` extraction and triple generation. However, it sometimes struggled to extract triples at all, merely repeating the input text, which resulted in lower ROUGE scores. Combining the fine-tuned model with the guidance framework during inference proved not to offer significant advantages.

TABLE 7: Comparison of Model Performance on ROUGE Metrics: Evaluating Various Techniques and Models for Extracting Triples including the ENTITY TYPES from the Test Data. The highest value in each metric is in bold, with the second and third highest values underlined.

Technique	Model	ROUGE-1	ROUGE-2	ROUGE-3	ROUGE-6	ROUGE-L
PE one-shot	Llama 70B chat	0.1962	0.1433	0.1094	0.0534	0.1617
PE few-shot	Mistral 7B Instruct	0.5094	0.3593	0.2639	0.1240	0.3874
PE few-shot	Zephyr-7B-β	0.4986	0.3740	0.2873	0.1544	0.3824
Guidance few-shot	Llama 7B chat	<u>0.5292</u>	<u>0.4101</u>	<u>0.3205</u>	<u>0.1767</u>	<u>0.4090</u>
Guidance one-shot	Llama 13B chat	0.5044	0.3744	0.2856	0.1502	<u>0.4207</u>
Guidance one-shot	Llama 70B chat	**0.5565**	**0.4360**	<u>0.3479</u>	<u>0.2076</u>	**0.4372**
Guidance few-shot	Llama 70B chat	<u>0.5414</u>	<u>0.4134</u>	0.3193	0.1605	0.4018
FT txt	Llama 7B chat	0.4351	0.3973	**0.3623**	**0.2648**	0.3857
FT txt + Guidance	Llama 7B chat	0.5215	0.4031	0.3153	0.1752	0.4041

Based on these results, both the fine-tuned Llama 2 7B chat model and the 70B guidance model equipped with entity type extraction were chosen for generating the knowledge graph from the 12,000 documents. Consistent with the performance observed on the test data, the fine-tuned extracted fewer triples, yielding a total of 77,307. The qualitative assessment revealed that 55 out of 100 randomly sampled triples were correct, indicating significant improvement compared to fine-tuning and extraction processes that did not utilize entity types. Post-processing efforts such as enforcing the specified ontology, excluding `Time` entities lacking a date, omitting triples where the `Malware` entity type was mentioned fewer than five times across all documents, and removing non-named entities from the `Malware` and `ThreatActor` categories, effectively reduced the number to 41,525 triples. Although the data still contained some noise, the qualitative assessment showed improvement with 77 out of 100 triples being

correct. These refined triples were then used for link prediction. The guidance model extracted nearly 460,000 triples from the documents, but the qualitative assessment revealed that the majority were incorrect, with only 15 out of 100 random samples being correct. After post-processing, only 17,050 entities remained, demonstrating higher quality, as evidenced by 66 correct samples in the qualitative assessment. This significant reduction in triples was due to the model incorrectly connecting entity types with certain relationships and extracting triples with newly invented entity types or relationships. Given that the fine-tuned model yielded better-quality triples, the triples from the guidance model were not utilized for further link prediction tasks.

Finding 3: Applying models that perform well on a small-scale test set to a large-scale dataset can yield poor results with a lot of noise. To adhere to an ontology for triple extraction, including the entity types in the few-shot examples or fine-tuning data is necessary and produces output better suitable for post-processing.

4.4. Link Prediction (RQ4)

For evaluating the efficacy of the prediction task, especially within knowledge graph forecasting, a comparison with triples generated using LADDER, as presented by Alam et al. [3], based on the same corpus of 12,000 documents was conducted. Traditional metrics such as Mean Reciprocal Rank (MRR) and Hits@n were employed. These metrics derive from the rankings that the link prediction model assigns to all accurate test triples. MRR represents the mean of the inverse rankings of all correct test triples, while Hits@n indicates the proportion of instances where a true triple appears within the top n positions of the rankings. Higher values in these metrics indicate superior performance.

The evaluation utilized the TuckER [46] model within a transductive setting across two distinct test sets. *TestSet1* comprises hand-annotated triples that are not included in the training triples and have not been mapped to MITRE ATT&CK techniques. Given TuckER's reliance on pre-identified entities, an additional test set containing only entities that were extracted by both LADDER and the fine-tuned 7B chat model was constructed as *TestSet2*. TuckER was implemented using the *pykeen* library [63], adopting Alam et al.'s [3] parameters. The setup included 50 embedding dimensions, a batch size of 64, and an initial learning rate of 0.001, while the training was reduced to 500 epochs.

The results, as presented in Table 8, demonstrate that for *TestSet1*, the TuckER model, when trained on triples extracted through the fine-tuned 7B chat model, achieved significantly higher scores, with a notable increase of 9.26% in Hits@30 compared to its counterpart. For *TestSet2*, the performance gap narrows, with the fine-tuned model exhibiting only marginal improvements in Hits@30 and MRR metrics. This suggests that the triples extracted using our methodology may be slightly more refined, thereby enhancing the model's ability to predict links between known entities more effectively. However, when comparing our results to those reported by Alam et al. [3], it is evident that our overall scores are significantly lower. This discrepancy can be attributed to the absence of a mapping for MITRE ATT&CK techniques, which leads to a reduction in the number of unique entities, consequently simplifying both the training and prediction processes.

TABLE 8: Link Prediction Results with TuckER & NodePiece for LADDER and Llama 2 7B fine-tuned chat model, including for Tail, Head, and Both results. H@ is Hits@.

	Head				**Tail**				**Both**			
	H@3	H@10	H@30	MRR	H@3	H@10	H@30	MRR	H@3	H@10	H@30	MRR
Transductive Link Prediction TuckER TestSet-1												
LADDER	.0194	.0291	.0922	.0282	.0269	.0791	.1279	.0304	.0364	.1019	.1869	.0442
FT 7B	.0388	.0631	.1505	.0390	.0370	.1077	.2205	.0451	.0437	.1117	.2379	.0563
Transductive Link Prediction TuckER TestSet-2												
LADDER	.0135	.0202	.0303	.0132	.0534	.1748	.2816	.0601	.0202	.0497	.0791	.0218
FT 7B	.0522	.0875	.1481	.0530	.0485	.1602	.3252	.0735	.0446	.0976	.1843	.0491
Inductive Link Prediction NodePiece TestSet-3												
LADDER	.8545	1	1	.6397	.2909	.6545	1	.2442	.5727	.8273	1	.4419
FT 7B	.9273	1	1	.7594	.2909	.6545	.9636	.2447	.6091	.8273	.9818	.5020

Further inductive link prediction with the NodePiece [48] implementation in *pykeen* library [63] was conducted on an inference dataset containing entities not included in the test data. The results of the two models, one trained with the extracted triples from LADDER and one with triples from the fine-tuned 7B chat model, are very close. This implies that the relationships within the triples are fairly clear, enabling both models to effectively generalize from observed to unseen data. *For example*, the trained NodePiece is capable of inferring a new relationship between the malware `Android/AdDisplay.Ashas`, which was not included in the training data, and the `Android` operating system. It accurately predicts this relationship when queried with the malware as the head entity and `targets` as the relationship. The second most likely prediction for this relationship is `Russian`, reflecting the significant number of users in Russia who were affected by this malware [64].

Finding 4: The KG constructed from unstructured CTI using LLM exhibits promising link prediction capabilities, with models trained on extracted triples showing enhanced performance and robust generalization from known to unknown data relationships in certain scenarios.

5. Discussion

5.1. Case Study

In this section, we present additional case studies for a qualitative assessment and comparison of fine-tuning and guidance methods. As shown in the outputs from both models in Table 9, the fine-tuned 7B chat model confines its responses to malware-relevant triples and adheres to the specific ontology it was trained on during the fine-tuning process. On the other hand, the 70B chat model guided by prompts encounters difficulties in this respect. For instance in the first example, it incorrectly associates the entity type `MalwareType` with the relationships `targets`, `isA`, or `uses` in connection to `MalwareType` and `Person`, or assigns the type `Organization` the relationship `hasAuthor` with `Malware Type`. Additionally, it extracts triples related to the MITRE ATT&CK framework mentioned in the paragraph, which are irrelevant to the malware discussion.

The same issue is apparent in the second example, where the model incorrectly extracts the irrelevant triple `[Trend Micro[Organization], hasAuthor, Danielle Walker[Person]]` and mistakenly links the type `Organization` with `indicates` to `AttackPattern`. However, in

TABLE 9: Case Study: A Comparison of the Llama 2 7B Fine-tuned Model and the Llama 2 70B Guidance Model in Extracting Triples from Two Example Paragraphs of Unstructured CTI Reports.

Paragraph	7B chat Fine-tuned	70B chat Guidance
Leak of Alleged Pegasus Target List Restokes Spyware Debate Pegasus Spyware Critics See Shift to Mass Surveillance; Vendor NSO Group Disagrees Amnesty International on Sunday warned that Pegasus software was able to exploit Apple's latest iPhone 12 running the current 14.6 operating system. (Photo: Apple) Allegations that commercially available spyware is being abused by countries to spy on dissidents, journalists, political rivals and business leaders are again in the spotlight. See Also: Live Webinar — Improve Cloud Threat Detection and Response using the MITRE ATT&CK Framework The leaking of an alleged target list tied to users of Israel-based NSO Group's Pegasus spyware - built to infect even the latest, fully patched Apple and Android devices - has led to questions over the scale of such operations, if the use of Pegasus gets sufficiently policed and whether the sale of spyware to certain countries should be blocked.	`[Pegasus[Malware], hasAuthor, Israel-based NSO Group[ThreatActor]], [Pegasus[Malware], targets, Apple[Organization]], [Pegasus[Malware], targets, iPhone 12[Hardware]], [Pegasus[Malware], targets, Android[OS]]`	`[Pegasus[MalwareType], isA, Spyware[MalwareType]], [Pegasus[MalwareType], targets, dissidents[Person]], [Pegasus[MalwareType], targets, journalists[Person]], [Pegasus[MalwareType], targets, political rivals[Person]], [Pegasus[MalwareType], targets, business leaders[Person]], [Pegasus[MalwareType], uses, exploit Apple's latest iPhone 12 running the current 14.6 operating system[AttackPattern]], [NSO Group[Organization], hasAuthor, Pegasus spyware[MalwareType]], [NSO Group[Organization], uses, built to infect even the latest, fully patched Apple and Android devices[AttackPattern]], [MITRE ATT&CK Framework[Indicator], isA, Live Webinar[Event]], [MITRE ATT&CK Framework[Indicator], isA, Improve Cloud Threat Detection and Response[Event]]`
by Danielle Walker, Senior Reporter July 22, 2014 Op Emmental spoofs bank sites, and uses Android malware to maintain account access So far, Operation Emmental has targeted victims' accounts in Austria, Switzerland, Sweden, and Japan. Cybercriminals have crafted an intricate attack method for maintaining a foothold in victim's online bank accounts, researchers warn. On Tuesday, Trend Micro released a 20-page report (PDF) on "Operation Emmental," which makes use of Android malware capable of beating two-factor authentication and also delivers malicious code that changes an infected computer's DNS settings so it points to attacker-operated servers. Saboteurs begin the scheme by delivering malware through phishing attacks – malicious links or attachments designed to look like correspondence from popular retailers	[Operation Emmental[ThreatActor], targets, Austria[Location]], [Operation Emmental[ThreatActor], targets, Switzerland[Location]], [Operation Emmental[ThreatActor], targets, Sweden[Location]], [Operation Emmental[ThreatActor], targets, Japan[Location]]	[Operation Emmental[ThreatActor], targets, Austria[Location]], [Operation Emmental[ThreatActor], targets, Switzerland[Location]], [Operation Emmental[ThreatActor], targets, Sweden[Location]], [Operation Emmental[ThreatActor], targets, Japan[Location]], [Operation Emmental[ThreatActor], uses, Android malware[MalwareType]], [Operation Emmental[ThreatActor], uses, malicious code[MalwareType]], [Operation Emmental[ThreatActor], uses, phishing attacks[AttackPattern]], [Operation Emmental[ThreatActor], uses, malware capable of beating two-factor authentication[AttackPattern]], [Operation Emmental[ThreatActor], discoveredIn, July 22, 2014[Time]], [Trend Micro[Organization], hasAuthor, Danielle Walker[Person]], [Trend Micro[Organization], hasAlias, Senior Reporter[Person]], [Trend Micro[Organization], indicates, Operation Emmental has targeted victims' accounts[AttackPattern]], [Trend Micro[Organization], discoveredIn, July 22, 2014[Time]]

this case, the guidance model successfully extracts additional pertinent information, such as `[Operation Emmental[ThreatActor], uses, malware capable of bypassing two-factor authentication[AttackPattern]]`, which the fine-tuned model overlooks.

Despite the challenges, after filtering out triples that do not adhere to the ontology, the fine-tuned model ultimately produced a larger number of relevant triples, indicating that the guidance model's issues with relevance and accuracy have a more significant impact.

5.2. Limitations and Future Work

The study highlighted areas for potential improvement. Given the trial-and-error nature of prompt development and the vast range of possibilities, there is always the potential for discovering more effective prompts that could lead to enhanced model performance for guidance or the fine-tuning method. Additionally, the scope of annotated data available for fine-tuning was limited; access to a larger annotated dataset would likely enhance the model's accuracy, reduce bias, and improve generalizability in the fine-tuned model. Refining data pre-processing techniques could further improve results, particularly by removing clearly irrelevant data, as models still appear to struggle with distinguishing such content. Moreover, the challenge of employing link prediction without attack pattern mapping was amplified by the extensive variety of potential connections, indicating areas for future enhancements.

6. Conclusion

This research demonstrates the effectiveness of using Large Language Models (LLMs) and Knowledge Graphs (KGs) for automating the extraction of actionable Cyber Threat Intelligence (CTI) from unstructured data. It highlights the impact of various extraction methodologies, finding that guidance framework significantly improved the output beyond what is achievable with few-shot prompt engineering alone. Furthermore, the study underlines the importance of fine-tuning for extracting triples that adhere to a specified ontology. While the approach shows promise in structuring vast amounts of CTI and aiding in threat understanding, challenges remain in scaling and refining the methods for broader application. This study lays the groundwork for future advancements in automated CTI processing using modern technologies, indicating a significant leap forward in cybersecurity defenses.

Acknowledgments

As part of the open-report model followed by the Workshop on Attackers & CyberCrime Operations (WACCO), all the reviews for this paper are publicly available at https://github.com/wacco-workshop/WACCO/tree/main/WACCO-2024.

References

[1] R. Brown and K. Nickels, "SANS 2023 CTI Survey: Keeping Up with a Changing Threat Landscape," 2023. [Online]. Available: https://www.sans.org/white-papers/2023-cti-survey-keeping-up-changing-threat-landscape

[2] N. Rani, B. Saha, V. Maurya, and S. K. Shukla, "TTPHunter: Automated Extraction of Actionable Intelligence as TTPs from Narrative Threat Reports," in *Australasian Information Security Conference (AISC 2023)*. ACM, 2023, pp. 126–134.

[3] M. T. Alam, D. Bhusal, Y. Park, and N. Rastogi, "Looking Beyond IoCs: Automatically Extracting Attack Patterns from External CTI," in *The 26th International Symposium on Research in Attacks, Intrusions and Defenses (RAID '23)*, 2023.

[4] Wang et al., "GPT-NER: Named Entity Recognition via Large Language Models," 2023. [Online]. Available: http://arxiv.org/abs/2304.10428

[5] Wan et al., "GPT-RE: In-context Learning for Relation Extraction using Large Language Models," 2023. [Online]. Available: http://arxiv.org/abs/2305.02105

[6] S. Wadhwa, S. Amir, and B. Wallace, "Revisiting Relation Extraction in the era of Large Language Models," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 2023, pp. 15 566–15 589.

[7] Zhu et al., "LLMs for Knowledge Graph Construction and Reasoning: Recent Capabilities and Future Opportunities," 2023. [Online]. Available: http://arxiv.org/abs/2305.13168

[8] J. Han, N. Collier, W. Buntine, and E. Shareghi, "PiVe: Prompting with Iterative Verification Improving Graph-based Generative Capability of LLMs," 2023. [Online]. Available: http://arxiv.org/abs/2305.12392

[9] M. Trajanoska, R. Stojanov, and D. Trajanov, "Enhancing knowledge graph construction using large language models," 2023. [Online]. Available: http://arxiv.org/abs/2305.04676

[10] R. Fayyazi and S. J. Yang, "On the Uses of Large Language Models to Interpret Ambiguous Cyberattack Descriptions." [Online]. Available: http://arxiv.org/abs/2306.14062

[11] V. Jüttner, M. Grimmer, and E. Buchmann, "ChatIDS: Explainable Cybersecurity Using Generative AI," 2023. [Online]. Available: http://arxiv.org/abs/2306.14504

[12] M. Gupta, C. Akiri, K. Aryal, E. Parker, and L. Praharaj, "From ChatGPT to ThreatGPT: Impact of Generative AI in Cybersecurity and Privacy," 2023. [Online]. Available: http://arxiv.org/abs/2307.00691

[13] Touvron et al., "Llama 2: Open foundation and fine-tuned chat models," 2023. [Online]. Available: http://arxiv.org/abs/2307.09288

[14] Jiang et al., "Mistral 7b," 2023. [Online]. Available: http://arxiv.org/abs/2310.06825

[15] Tunstall et al., "Zephyr: Direct distillation of LM alignment," 2023. [Online]. Available: http://arxiv.org/abs/2310.16944

[16] OpenAI, "Prompt engineering - OpenAI Platform." [Online]. Available: https://platform.openai.com/docs/guides/prompt-engineering

[17] S. Lundberg, H. Nori, and M. T. Ribeiro, "guidance-ai/guidance." [Online]. Available: https://github.com/guidance-ai/guidance

[18] Hu et al., "LoRA: Low-rank adaptation of large language models," 2021. [Online]. Available: http://arxiv.org/abs/2106.09685

[19] C. S. Johnson, M. L. Badger, D. A. Waltermire, J. Snyder, and C. Skorupka, "Guide to Cyber Threat Information Sharing," 2016, NIST SP 800-150. [Online]. Available: https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-150.pdf

[20] Symantec, "Symantec Enterprise Blogs/Threat Intelligence," 2024. [Online]. Available: https://symantec-enterprise-blogs.security.com/blogs/threat-intelligence

[21] Mandiant, "Threat intelligence reports," 2024. [Online]. Available: https://www.mandiant.com/resources/reports

[22] CrowdStrike, "CrowdStrike Blog," 2024. [Online]. Available: https://www.crowdstrike.com/blog

[23] H. Dalziel, *How to define and build an effective cyber threat intelligence capability*. Syngress, 2014.

[24] G. Husari, E. Al-Shaer, M. Ahmed, B. Chu, and X. Niu, "TTP-Drill: Automatic and Accurate Extraction of Threat Actions from Unstructured Text of CTI Sources," in *Proceedings of the 33rd Annual Computer Security Applications Conference*. ACM, 2017, pp. 103–115.

[25] Z. Zhu and T. Dumitras, "ChainSmith: Automatically Learning the Semantics of Malicious Campaigns by Mining Threat Intelligence Reports," in *2018 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 2018, pp. 458–472.

[26] T. Satyapanich, F. Ferraro, and T. Finin, "CASIE: Extracting Cybersecurity Event Information from Text," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 5, 2020, pp. 8749–8757.

[27] E. Aghaei, X. Niu, W. Shadid, and E. Al-Shaer, "SecureBERT: A domain-specific language model for cybersecurity," 2022. [Online]. Available: http://arxiv.org/abs/2204.02685

[28] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," 2019. [Online]. Available: https://arxiv.org/abs/1810.04805

[29] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "RoBERTa: A Robustly Optimized BERT Pretraining Approach," 2019. [Online]. Available: https://arxiv.org/pdf/1907.11692

[30] Vaswani et al., "Attention is All you Need," in *31st Conference on Neural Information Processing Systems (NIPS 2017), Long Beach, CA, USA.*, 2017.

[31] Zhao et al., "A Survey of Large Language Models," 2023. [Online]. Available: http://arxiv.org/abs/2303.18223

[32] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient Finetuning of Quantized LLMs," 2023. [Online]. Available: http://arxiv.org/abs/2305.14314

[33] Hugging Face. LLM prompting guide. [Online]. Available: https://huggingface.co/docs/transformers/main/tasks/prompting

[34] The MITRE Corporation, "MITRE ATT&CK®," 2024. [Online]. Available: https://attack.mitre.org

[35] X. Li, F. Polat, and P. Groth, "Do Instruction-tuned Large Language Models Help with Relation Extraction?" in *KBC-LM'23: Knowledge Base Construction from Pre-trained Language Models workshop at ISWC 2023*, 2023.

[36] K. Zhang, B. J. Gutiérrez, and Y. Su, "Aligning Instruction Tasks Unlocks Large Language Models as Zero-Shot Relation Extractors," 2023. [Online]. Available: http://arxiv.org/abs/2305.11159

[37] Siracusano et al., "Time for aCTIon: Automated Analysis of Cyber Threat Intelligence in the Wild," 2023. [Online]. Available: http://arxiv.org/abs/2307.10214

[38] A. Rossi, D. Firmani, A. Matinata, P. Merialdo, and D. Barbosa, "Knowledge Graph Embedding for Link Prediction: A Comparative Analysis," in *ACM Transactions on Knowledge Discovery from Data*, 2021, vol. 15, no. 2, pp. 1–49.

[39] E. Kiesling, A. Ekelhart, K. Kurniawan, and F. Ekaputra, "The SEPSES Knowledge Graph: An Integrated Resource for Cybersecurity," in *The Semantic Web – ISWC 2019*. Springer International Publishing, 2019, vol. 11779, pp. 198–214.

[40] L. Sun, Z. Li, L. Xie, M. Ye, and B. Chen, "APTKG: Constructing Threat Intelligence Knowledge Graph from Open-Source APT Reports Based on Deep Learning," in *2022 5th International Conference on Data Science and Information Technology (DSIT)*. IEEE, 2022, pp. 01–06.

[41] Z. Li, J. Zeng, Y. Chen, and Z. Liang, "AttacKG: Constructing Technique Knowledge Graph from Cyber Threat Intelligence Reports," 2022. [Online]. Available: http://arxiv.org/abs/2111.07093

[42] Gao et al., "ThreatKG: A Threat Knowledge Graph for Automated Open-Source Cyber Threat Intelligence Gathering and Management," 2022. [Online]. Available: http://arxiv.org/abs/2212.10388

[43] N. Rastogi, S. Dutta, M. J. Zaki, A. Gittens, and C. Aggarwal, "TINKER: A framework for Open source Cyberthreat Intelligence," in *2022 IEEE International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*, 2022.

[44] Z. Sun, Z. Deng, J. Nie, and J. Tang, "RotatE: Knowledge Graph Embedding by Relational Rotation in Complex Space," *CoRR*, vol. abs/1902.10197, 2019. [Online]. Available: http://arxiv.org/abs/1902.10197

[45] X. Jiang, Q. Wang, and B. Wang, "Adaptive Convolution for Multi-Relational Learning," in *North American Chapter of the Association for Computational Linguistics*, 2019.

[46] I. Balazevic, C. Allen, and T. Hospedales, "TuckER: Tensor Factorization for Knowledge Graph Completion," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Association for Computational Linguistics, 2019, pp. 5184–5193.

[47] Ali et al., "Improving Inductive Link Prediction Using Hyper-Relational Facts," 2021. [Online]. Available: https://arxiv.org/abs/2107.04894

[48] M. Galkin, E. Denis, J. Wu, and W. L. Hamilton, "NodePiece: Compositional and Parameter-Efficient Representations of Large Knowledge Graphs," 2022. [Online]. Available: https://arxiv.org/abs/2106.12144

[49] K. Liu, F. Wang, Z. Ding, S. Liang, Z. Yu, and Y. Zhou, "A review of knowledge graph application scenarios in cyber security," 2022. [Online]. Available: https://arxiv.org/abs/2204.04769

[50] OpenAI, "Best practices for prompt engineering with OpenAI API | OpenAI help center." [Online]. Available: https://help.openai.com/en/articles/6654000-best-practices-for-prompt-engineering-with-openai-api

[51] P. Rechia, "A Deep Dive Into Guidance's Source Code," 2023. [Online]. Available: https://betterprogramming.pub/a-deep-dive-into-guidances-source-code-16681a76fb20

[52] Taori et al., "Stanford Alpaca: An Instruction-following LLaMA model," https://github.com/tatsu-lab/stanford_alpaca, 2023.

[53] P. von Platen, “How to generate text: using different decoding methods for language generation with Transformers.” [Online]. Available: https://huggingface.co/blog/how-to-generate

[54] Hugging Face, “Text generation strategies.” [Online]. Available: https://huggingface.co/docs/transformers/main/generation_strategies

[55] ——, “Hugging Face Models,” 2024. [Online]. Available: https://huggingface.co/models

[56] ——, “Hugging Face Transformers,” 2024. [Online]. Available: https://github.com/huggingface/transformers

[57] T. Dettmers, “bitsandbytes,” 2024. [Online]. Available: https://github.com/TimDettmers/bitsandbytes

[58] AutoGPTQ, “AutoGPTQ,” 2024. [Online]. Available: https://github.com/AutoGPTQ/AutoGPTQ

[59] Hugging Face, “peft,” 2024. [Online]. Available: https://github.com/huggingface/peft

[60] ——, “trl,” 2024. [Online]. Available: https://github.com/huggingface/trl

[61] ——, “accelerate,” 2024. [Online]. Available: https://github.com/huggingface/accelerate

[62] C.-Y. Lin, “ROUGE: A Package for Automatic Evaluation of Summaries,” in *Annual Meeting of the Association for Computational Linguistics*, 2004.

[63] M. Ali, M. Berrendorf, C. T. Hoyt, L. Vermue, S. Sharifzadeh, V. Tresp, and J. Lehmann, “PyKEEN 1.0: A Python Library for Training and Evaluating Knowledge Graph Embeddings,” *Journal of Machine Learning Research*, vol. 22, no. 82, pp. 1–6, 2021.

[64] L. Stefanko, “ESET Research: Tracking down the developer of Android adware affecting millions of users,” 2019. [Online]. Available: https://www.welivesecurity.com/2019/10/24/tracking-down-developer-android-adware/